A SA-Based Floorplan Framwork for Fixed Outline Floorplanning: Maximizing Utilization and Minimizing Routing Length

Ching-Sheng Chang
Electrical Engineering
National Central University
Taoyuan, Taiwan
jonathan20150622@gmail.com

Tsai-Chen Li
Electrical Engineering
National Central University
Taoyuan, Taiwan
elaine00488400@gmail.com

Hsuan-Yu Weng
Electrical Engineering
National Central University
Taoyuan, Taiwan
eunice920526@gmail.com

Chia-Chien Cheng
Electrical Engineering
National Central University
Taoyuan, Taiwan
jessy20030503@gmail.com

***Abstract*—In chip design, floorplanning is vital for placing soft modules within fixed outlines while minimizing wirelength. The challenge lies in the fact that soft modules can deform and may not necessarily be arranged in a rectangular form within the chip. We need to determine what shapes would best minimize wirelength for the overall placement while also assessing whether these shapes adhere to VLSI design standards. To address this issue, we developed a Simulated Annealing (SA) algorithm [1], initially partitioning the chip into four quadrants using the Fiduccia-Mattheyses (FM) technique [2]. This enables individual quadrant optimization before applying SA globally. Customized cost functions and deformations enhance optimization, resulting in a more efficient circuit design.**

Keywords—Floorplanning, Simulated Annealing, Customized cost functions, Fiduccia-Mattheyses

I. Introduction

With technology advancing continuously, chip design complexity and transistor counts are on the rise. Hierarchical design and Intellectual Property (IP) modules are widely embraced to manage this complexity. Floorplanning is crucial in these methodologies, providing early-stage feedback on system architecture, estimating chip area, and predicting routing delays and congestion. Thus, it remains an essential component of VLSI circuit design, growing in importance over time.

Fig. 1 illustrates the general flow chart of the physical design process. In this paper, our focus will be on three key stages: Partitioning, Floorplanning, and Placement.

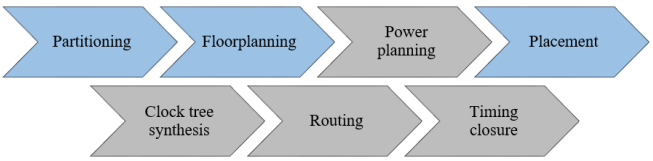


Fig. 1. Physical Design Flow chart.

Chip dimensions are predetermined in the initial design phase, leading to established fixed-outline constraints. Simply minimizing area is insufficient in modern integrated circuit design. By integrating floorplanning methods that consider fixed outlines, modules can be strategically placed to optimize space utilization and minimize connection lengths within the chip's boundary. As illustrated in Fig. 2, effective placement mitigates module overlapping, a critical consideration in contemporary IC design. Consequently, sophisticated placement algorithms are indispensable.

Also, total wirelength is another critical aspect of floorplanning. Excessive wirelength leads to system delays and increased power consumption. Thus, our focus in this study is to evaluate placements primarily based on the half-perimeter wirelength (HPWL). The HPWL is the sum of the Manhattan distance between each pair of modules multiplied by the number of connections between the two modules. The Manhattan distance between two modules is the distance between the centers of the minimum rectangles that completely cover the two modules.

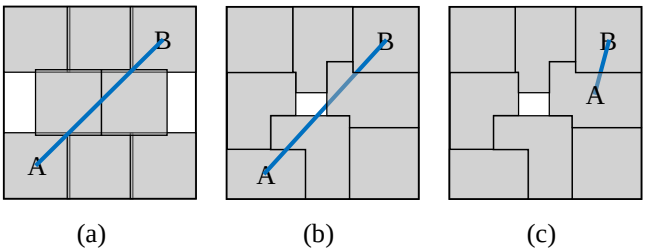


Fig. 2. The Example of (a) Non-Ideal and Illegal Placement. (b) Legal Placement with Longer HPWL. (c) Legal Placement with Shorter HPWL.

Prior studies in IC design primarily focused on minimizing chip area, yet traditional area-centric floorplanning methods are insufficient due to increasing complexity and fixed-outline constraints. Recent research, as discussed in [3], has explored techniques considering fixed outlines to enhance space utilization and reduce connection lengths. However, these approaches often overlook the interaction between fixed outlines and soft modules, posing a significant challenge in floorplanning optimization.

In [3], the paper addresses the challenge of placing modules within fixed outlines while adhering to constraints such as nonoverlapping modules and minimizing wire length. However, it exclusively considers fixed modules, neglecting the presence of soft modules. This highlights the need for a more comprehensive approach that seamlessly integrates considerations for both fixed outlines and soft modules to optimize chip designs effectively. Further exploration and advancement in VLSI circuit design are warranted to address this gap.

We have developed a set of algorithms using C++ programming language that efficiently places numerous fixed modules and soft modules within a predefined boundary, yielding favorable HPWL data.

We introduce a classification method using FM partitioning to divide the chip into four quadrants based on module

connections. Then, we utilize the SA algorithm with customized cost functions tailored to module placement characteristics. This allows for more effective exploration of the solution space, potentially resulting in shorter wirelength. Additionally, we employ deformations to bring modules closer together, achieving superior results in the global layout phase and enhancing circuit design efficiency.

Additionally, we've developed a visualization program using MATLAB to visually represent the floorplans generated by our algorithm and calculate their HPWL (Fig. 3). Based on the results, our algorithm consistently produces impressive outcomes for each unique case. In some instances, we even achieved better results than those officially released by the ICCAD.

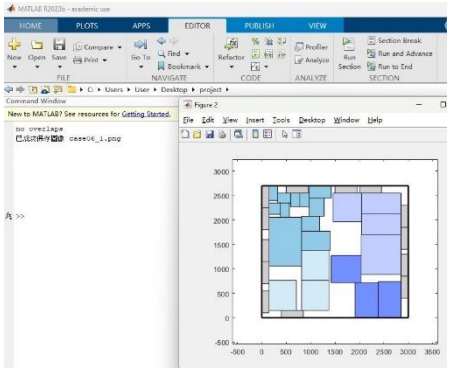


Fig. 3. Visualization of One of the Result Using MATLAB.

II. PREREQUISITE KNOWLEDGE

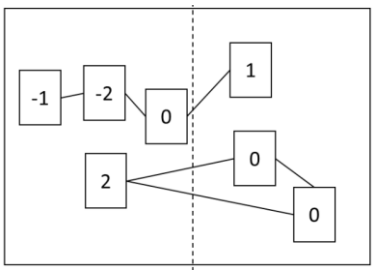
Below are some techniques or algorithms that we will use in this paper. We provide a brief overview here:

A. *Fiduccia-Mattheyses*

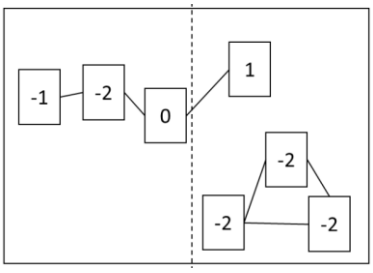
Fiduccia-Mattheyses (FM) algorithm is a widely used heuristic algorithm for partitioning graphs, often employed in VLSI design. The algorithm aims to divide the vertices of a graph into two sets, optimizing a certain objective function such as minimizing the number of edges between the two sets.

Here's a simplified overview of the FM algorithm:

1. Initialization: The FM algorithm starts by partitioning the vertices of the graph into two initial sets. This partitioning is often done randomly or based on predefined criteria. For instance, as illustrated in Fig. 4, each block represents a module, with lines between blocks indicating connections, and numbers denoting the gain achieved by moving a module from one set to the other. The gain is typically calculated based on the change in the objective function (e.g., the decrease in the number of edges cut).
2. Iterative Improvement: In each iteration, FM algorithm selects a vertex and evaluates the gain. The vertex with the highest gain is moved to the other set.
3. Termination: The algorithm continues iterating until no further improvement can be made, or until a stopping criterion is met (e.g., a maximum number of iterations).
4. Finalization: After the algorithm terminates, the resulting partition is obtained, with the vertices grouped into two sets based on the final assignments.



(a)



(b)

Fig. 4. FM Algorithm. (a) Initial State. (b) FM process.

The FM algorithm is efficient and effective for graph partitioning problems, making it a popular choice in various fields, including circuit design and network optimization.

B. *Simulated Annealing*

The Simulated Annealing (SA) algorithm is a probabilistic optimization technique inspired by the annealing process in metallurgy. SA is commonly used to find near-optimal solutions to combinatorial optimization problems (Fig. 5).

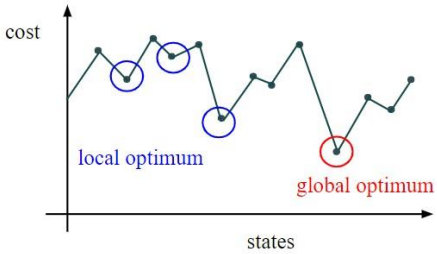


Fig. 5. Diagram of SA Process.

Here's a simplified overview of the SA algorithm:

1. Initialization: Start with an initial solution to the optimization problem.
2. Iterative Improvement: In each iteration, the algorithm perturbs the current solution by making a small random change. It then evaluates the quality of the new solution using an objective function (cost function).
3. Acceptance Criterion: If the new solution is better than the current solution, it is always accepted. However, if the new solution is worse, it may still be accepted probabilistically. The probability of accepting a worse solution decreases as the algorithm progresses and is influenced by a parameter called the "temperature."
4. Cooling Schedule: The temperature parameter decreases gradually over time according to a predefined schedule. As the temperature decreases, the probability of accepting worse solutions decreases as well.

This process mimics the cooling of a material in metallurgy.

5. Termination: The algorithm continues iterating until a stopping criterion is met, such as reaching a maximum number of iterations or achieving a satisfactory solution.

The SA algorithm is known for its ability to escape local optima and explore a wider solution space compared to deterministic methods. It has applications in various fields, including optimization problems in engineering, logistics, and machine learning.

C. *B*tree*

The B*tree [4] is a self-balancing tree data structure that maintains sorted data and allows for efficient insertion, deletion, and search operations. It's an extension of the B-tree data structure, designed specifically for use in external memory or secondary storage systems. A B*tree consists of nodes with a variable number of keys and pointers to child nodes. Fig. 6 is a placement using B*tree.

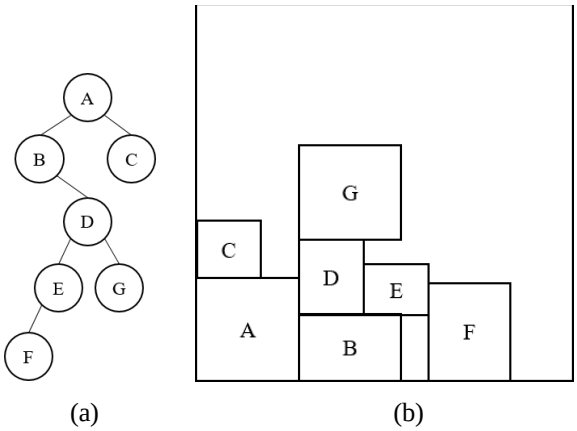


Fig. 6. An Example of B*tree. (a) Example B*tree. (b) Placement Corresponding to this B*tree.

B*trees are commonly used in databases and file systems where large amounts of data need to be stored and efficiently accessed. They provide fast search, insertion, and deletion operations, making them suitable for managing indexes and organizing data in secondary storage.

D. *Linear Ordering Algorithm*

The linear ordering algorithm [5] is a method used to arrange the vertices of a graph into a linear sequence, such that the sequence reflects the partial order of the vertices induced by the edges of the graph. This linear arrangement is also known as a topological ordering in the case of directed acyclic graphs (DAGs).

Here's a simplified overview of the linear ordering algorithm:

1. Initialization: Start with an empty linear sequence.
2. Traversal: Traverse the graph, selecting a vertex with no incoming edges (in-degree equals zero) as the starting point.
3. Selection: Add the selected vertex to the linear sequence and remove it from the graph.
4. Update: Update the in-degrees of the remaining vertices in the graph.
5. Repeat: Repeat steps 2-4 until all vertices have been added to the linear sequence.

The linear ordering algorithm is commonly used in various applications such as scheduling tasks, resolving dependencies, and optimizing code generation in compilers. It ensures that vertices are processed in the correct order according to their dependencies, thus avoiding cycles and ensuring efficient execution.

III. Problem Statement

In our research, we use HPWL as the optimization objective. Considering the layout configuration of soft and hard modules on the chip, our goal is to develop a planar placer that ensures no overlap between soft modules, as well as between soft and fixed modules. The optimization objective of this planar placer is measured based on HPWL.

During the initial layout planning, we adhere to the following crucial assumptions:

- Rectangular Shape of Soft Modules: Simplifies the complexity of layout planning.
- Aspect Ratio (Height-to-Width Ratio) of Soft Modules is 0.5~2: Ensures design balance and visual neatness.
- Soft Module Area Utilization is 80%~100%: Reduces blank areas on the chip.
- Corner Coordinates of Soft Module Positions are 0 or Positive Integers: Ensures layout tidiness.

Fig. 7 presents examples of invalid shapes for soft modules. In Fig. 7 (a), the region enclosed by the polygon contains a hole. In Fig. 7 (b), the region enclosed by the polygon exceeds one. Fig. 7 (c) violates the aspect ratio restriction (5/2=2.5, outside the range of 0.5 to 2), while Fig. 7 (d) violates the soft module area utilization restriction (18/30=60%, outside the range of 80% to 100%).

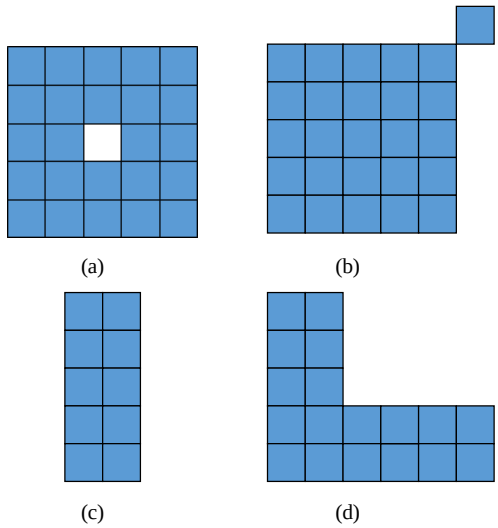


Fig. 7. Examples of Invalid Module Shapes. (a) The Region Enclosed by the Polygon Contains a Hole. (b) More than One Region Enclosed by the Polygon. (c) Violates the Aspect Ratio Restriction. (d) Violates the Soft Module Area Utilization Restriction.

Our planar placer, considering these assumptions, aims to find a reasonable layout to optimize the HPWL. This involves intelligent adjustments to the positions, shapes, and layouts of soft modules. In the following sections, we will delve into the detailed steps of the proposed initial layout planning method and its application effectiveness in real-world cases.

IV. RESEARCH METHOD

Fig. 8 illustrates the general flow of our design process. In the initialization phase, we first attempt to place soft modules within the fixed outline to obtain an initial solution. This process involves using the FM partitioning and Linear Ordering algorithms to acquire a favorable initial solution for subsequent SA optimization. Additionally, we have developed a Quad-Btree structure to store the relative positions of modules. Following this, we employ Fast-SA to perturb soft modules in an effort to obtain the optimal solution with the shortest HPWL. The details of each step will be elaborated in the subsections.

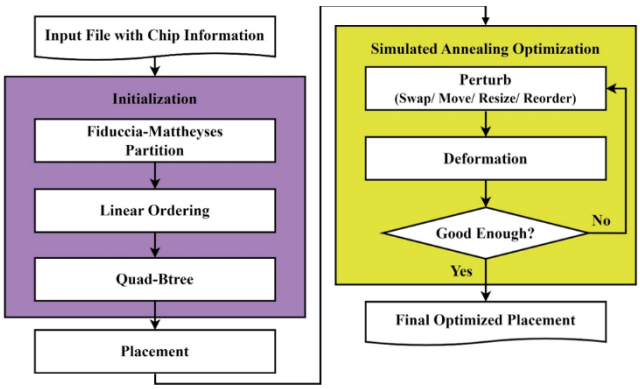


Fig. 8. Design Flow Chart

A. *Fiduccia-Mattheyses*

If there are numerous connections between modules, placing these modules nearby helps to shorten the total length of the bus. Therefore, we first divide the entire chip area into four sub-areas: upper left, upper right, lower right, and lower left by making a horizontal and vertical cut from the center point of the chip. Then, we classify the soft modules to determine the close connections between them and the relationship between the soft modules and the fixed modules.

Initially, we divide the entire layout into two parts based on the connections between each modules (Fig. 9). Then, using the FM algorithm based on the connections, we find a min-cut set. After two runs of the FM algorithm, the entire chip is successfully divided into four sub-regions (bottom-left: LD, top-left: LU, top-right: RU, bottom-right: RD). This placement and cutting strategy help optimize the chip layout, making the connections between soft modules more tightly knit.

B. *The Quad-Btree Representation*

The original B*tree placement method initially fixes the x-coordinate of a newly placed module, then adjusts its y-coordinate to confirm its position. However, because of the constraints imposed by the fixed positions of fixed modules and the subsequent complexity in perturbation, it becomes challenging to incorporate fixed modules into the B*tree. Additionally, as we divide the entire chip into four sub-regions and start placing soft modules from the four corners, gradually moving towards the center of the chip, this approach often leads to modules exceeding the boundaries, thus failing to meet the requirements of the original B*tree placement method. Therefore, by modifying the original B*tree so that new modules can simultaneously adjust their x and y coordinates to find a suitable placement and achieve a tighter packing, we have developed a new tree structure called the "Quad-Btree."

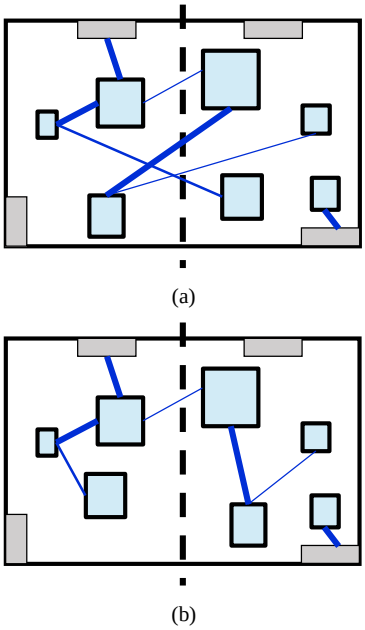


Fig. 9. Illustration of the FM Partitioning Based on the Number of Connections Between Soft Modules and Fixed Modules. (a) Initial State (b) FM Process.

We construct separate trees for each of the four sub-regions. To obtain a better initial solution, we utilize the results of the Linear Ordering Algorithm to determine the initial order of the Quad-Btree.

Our Quad-Btree structure originates from the four corners of the floorplan and models module positions using a depth-first search procedure. For instance, in the LD (bottom-left) sub-region, the root module of the Quad-Btree would be placed at the bottom-left corner. We recursively construct the left subtree and then the right subtree. As shown in Fig. 10 (a) and (b), if node B is the left child of node A, module B must be located to the right-hand side and adjacent to module A. On the other hand, if node C is the right child of node A, module C must be located above module A; contour is simultaneously generated to enclose the placed modules, facilitating subsequent placement. The same principle applies to all the other sub-regions. The detailed placement will be elucidated in the following subsection.

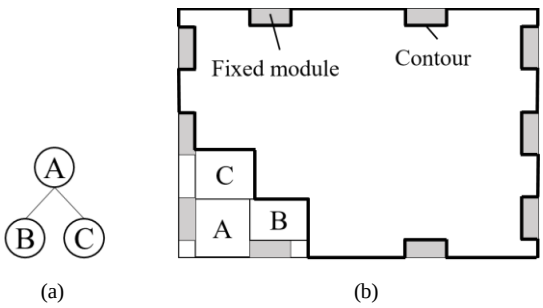


Fig. 10. An Example Placement Using Quad-Btree (a) Example Quad-Btree (b) Corresponding Placement

By employing the Quad-Btree representation, we transform the complex placement of modules into relative positions, enabling tight module placement. Additionally, using the Quad-Btree allows for easier execution of module position exchanges, facilitating subsequent operations.

C. Linear Ordering Algorithm

After that we utilize the Linear Ordering Algorithm to determine the sequential arrangement of modules with the objective of minimizing the overall wirelength of connections between modules. Incident nets of a given block are categorized into two types:

- Terminating nets have no other incident blocks that are unplaced.
- New nets have no pins on any block from the partially constructed ordering.

The algorithm prioritizes selecting the block with the highest difference between the number of terminating and new nets as the next block in the sequence. The algorithm begins by choosing an initial block, either arbitrarily or based on the number of connections to other blocks.

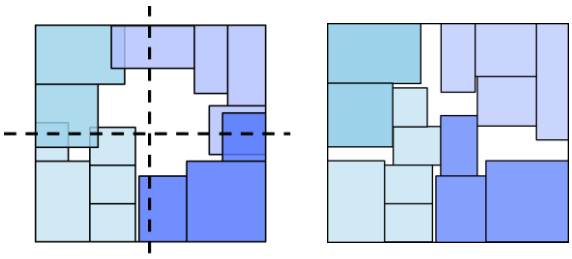
During each iteration:

- Gain (difference between terminating and new nets) is calculated for each block in set M.
- Blocks with the maximum gain are selected.
- If multiple, those with the most terminating nets are chosen.
- If there are still multiple blocks, those with the lowest connectivity are chosen.
- If there are still multiple blocks, one is selected arbitrarily.

The selected block is added to the order and removed from set M.

D. Placement using Quad-Btree

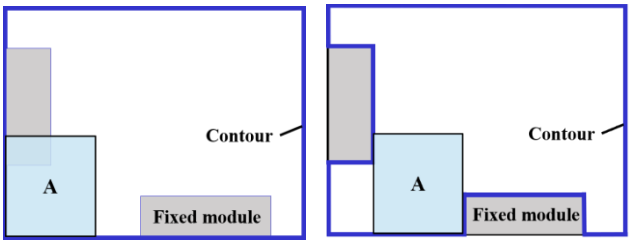
After sections A, B, and C, we will have four Quad-Btree structures to handle different sub-regions. However, there are limitations to placing each sub-region (top-left, top-right, bottom-right, and bottom-left) independently. Firstly, to meet the requirements of fixed outlines, all modules must be placed within the given area, which is overly constrained as there are no clear boundaries between regions. Additionally, there may be large modules that cannot fit within their designated areas, leading to significant module overlaps, see Fig. 11. Therefore, achieving an ideal placement may be challenging due to the lack of interaction between different regions. Instead, a better approach is to simultaneously optimize all modules and consider the interactions between different regions globally.



(a) (b)

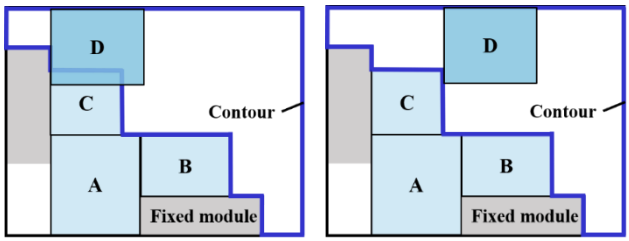
Fig. 11. Local Placement and Global Placement. (a) Place by Four Independent Region. (b) Place by Undivided Chip.

The above placement method can result in two potential overlap scenarios: overlap between soft modules and fixed modules, and overlap between soft modules from different sub-regions. Since the positions of fixed modules are fixed, to avoid overlap between soft modules and fixed modules, the fixed modules are first updated to fit within the contour before placement. If, after determining the placement based on the contour, there is still overlap with other fixed modules contained within the contour, the x-coordinate is shifted until there is no overlap (Fig. 12 and Fig. 13). For overlaps between soft modules from different sub-regions, the placement position is checked to see if it extends beyond the contour of another sub-region. If it does, indicating overlap with soft modules from other sub-regions, the x-coordinate is similarly shifted until there is no overlap. It is important to note that since the starting points for placement are the corners of the chip, the direction of x-coordinate shift varies depending on the sub-region. Fig. 14 presents our pseudocode for the above content.



(a) (b)

Fig. 12. Illustration of Updating Contour. (a) Place Module A by Contour with Overlap. (b) Update Fixed Modules in Contour.



(a) (b)

Fig. 13. Illustration of Moving Soft Module Based on Contour. (a) Place Module D by Contour with Overlap. (b) Move X-Coordinate of Module D Based on Contour.

Algorithm 1 Module Movement

```
function MOVEMENT
    if module is root_block then
        Set corner coordinates to corner of the chip
    else
        Calculate x_start, y_start by module's parent node
    end if
    Calculate x_end, y_end by pre-determined height and
width of module
    Determine x_max by contour within loop
    Determine y_max by contour within loop
    //Check overlap and adjust position
    while overlap do
        calualte x_move by contour
    end while
    x_start += x_move
    Determine y_max by contour within loop
    Update contour
end function
```

Fig. 14. Pseudocode for Shifting Soft Module Bases on Contour.

Taking the LD (button-left) region as an example, after updating the contour based on the given fixed modules, the Quad-Btree defined in section B can be directly applied to the

LD region. However, for other regions, the placement direction of x and y coordinates needs to be adjusted based on the contour of the region. For instance, in the RU (top-right) region, a starting point in the top-right corner can be determined based on the contour to apply the Quad-Btree placement method, moving left and down to place modules(Fig. 15).

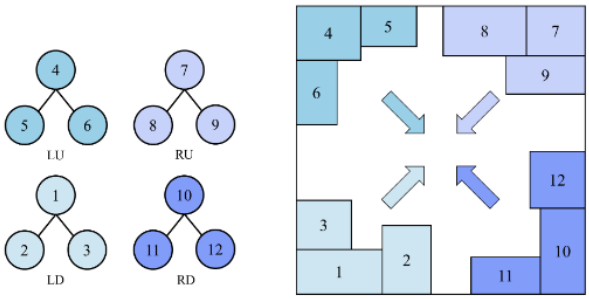


Fig. 15. An Example Placement Using Quad-Btree. (a)The Quad-Btree. (b)The Placement Corresponding to This Quad-Btree.

E. Fast Simulated-Annealing

With the initial solution in place, optimization of the layout can be achieved through the linear iterative algorithm of the Quad-Btree, aiming to place modules within the chip outline and minimize wirelength. This iterative process helps in finding the optimal solution. The primary advantage of SA is its capability to escape from local minimum and converge towards a global minimum. We use Fast-SA, instead of Classical-SA, for better performance.

The following is our four types of perturbation operations for the Quad-Btree used in Fast-SA:

- Op1: Exchange two nodes within the same subtree.
- Op2: Move a node within a subtree to another position.
- Op3: Adjust the size of a module.
- Op4: Change the order of sub-region placements.

For Op1, we rotate a soft module within a subtree. For Op2, we select a node from a subtree and move it to another position within the same subtree. For Op3, we alter the shape of a soft module, changing its length and width. We adjust the occurrence probabilities of each operation according to the SA process. In the first third of the total time, Op1 and Op2 account for 70% to 90% of occurrences, while in the second third, Op3 accounts for 70% to 90%. In the final third, Op1 to Op4 each account for 25% of occurrences. It is important to note that Op1, Op2, and Op3 do not affect the structure of the four-tree. For Op4, we randomly change the order of sub-region placements while keeping the internal structure of the trees unchanged.

Considering the various characteristics such as the size of fixed modules, the size of soft modules, the number of soft modules, and the ratio of white space, the parameters of the cost function will influence the optimal results. If the white space ratio is small, priority must be given to module size to avoid situations where modules cannot fit within the chip outline. However, if the white space ratio is large, module size will no longer be the primary focus of optimization, and the weight of total wirelength in the cost function will increase. Therefore, unlike previous studies, we will choose different cost functions based on the characteristics of module placement. Detailed experimental data and results will be presented in Section V.

To evaluate the quality of the macro placement scheme, the cost function F for macro placement is defined as follows:

$$\mathrm{F} = \alpha \frac{W}{W_{norm}} + (1-\alpha)P \tag{1}$$

Where W is the current HPWL, W_{norm} is the average of HPWL; P represents the penalty factor, which is the sum of the excess length or width exceeding the chip's boundary; α is the user-defined parameter. This cost function will be normalized between 0 and 1.

Before starting the Simulated Annealing process, we disturb the initial placement for a specific duration to calculate the average of HPWL, which becomes the normalized wirelength W_{norm}. Compared to the past, where minimizing area was typically a goal and area was often part of the cost function, our objective is simply to ensure modules fit within a fixed outline to minimize HPWL. Even if the area is not minimized, we only consider the excess beyond the boundary as a penalty factor to achieve legality. Furthermore, we can adjust the weight of the penalty factor by modifying the α parameter based on different input data of chip information, such as white-space ratio, to enhance the success probability of placement within the fixed outline and reduce the occurrence of infeasible solutions.

After that, we utilize the way of Fast-SA [6] to setting the temperature. The aim is to minimize the occurrence of uphill moves during the initial phase, as they typically lead to less desirable solutions. To achieve this, a more focused, greedy search strategy is employed initially to quickly converge towards a local optimum.

In simpler terms, Fast-SA operates in three main stages:

1) High-temperature random search stage:
   Initially, the algorithm explores widely by setting the temperature very high, favoring the acceptance of even inferior solutions to avoid getting trapped in a local optimum.
2) Pseudo-greedy local-search stage:
   As the search progresses, the temperature decreases gradually, allowing the algorithm to become more selective in accepting solutions. This stage emphasizes refining solutions towards a local optimum, accepting only a limited number of inferior solutions.
3) Hill-climbing search stage:
   Finally, the algorithm focuses on further improving solution quality by raising the temperature. This stage facilitates the search for better solutions, ultimately leading to convergence towards the desired solution.

The three stages over the search time are defined as follows.

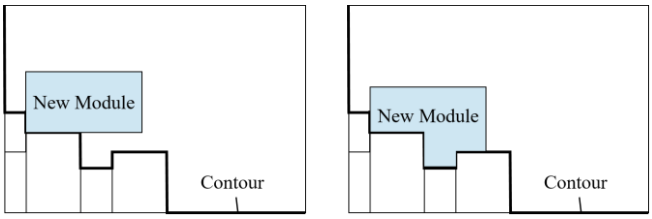
$$\mathrm{T}_r = \begin{cases} \frac{-\Delta avg}{\ln P}, & r = 1 \\ \frac{T_1 \Delta cost}{rc}, & 2 \le r \le k \\ \frac{T_1 \Delta cost}{r}, & r \ge k \end{cases}, \tag{2}$$

Where the value P is 0.95, c is 50, k is 30, and r is the number of iteration.

F. Deformation

Upon finalizing the selection of the appropriate Quad-Btree structure, the soft modules will be systematically integrated into the chip, adhering to the prescribed order of the Quad-Btree. Concurrently, efforts will be made to tailor the shapes of the soft modules being placed, transcending the constraints of rectangular forms, with the objective of closely aligning them with the underlying contours. This strategic adjustment aims to minimize white space between soft modules within a given block, thereby reducing the HPWL associated with internal connections within said block, while also mitigating potential instances of overlap or boundary infringements.

Initially, a thorough examination will be conducted to ascertain any gaps between the module and the contour. Should such gaps be identified, adjustments will be made by extending the module towards the contour (Fig. 16), ensuring alignment. Subsequently, meticulous fine-tuning will be applied to uphold the prescribed aspect ratio and area utilization of the soft module, thus ensuring compliance.



(a) (b)

Fig. 16. Deformation. (a) Original Placement. (b) Placement After Deformation.

In summary, we utilized FM, Fast-SA, and the Linear Ordering Algorithm, alongside the development of Quad-Btree and deforming algorithms. These methodologies enabled us to place all modules within a fixed outline while achieving a reduced HPWL.

V. Experimental Results

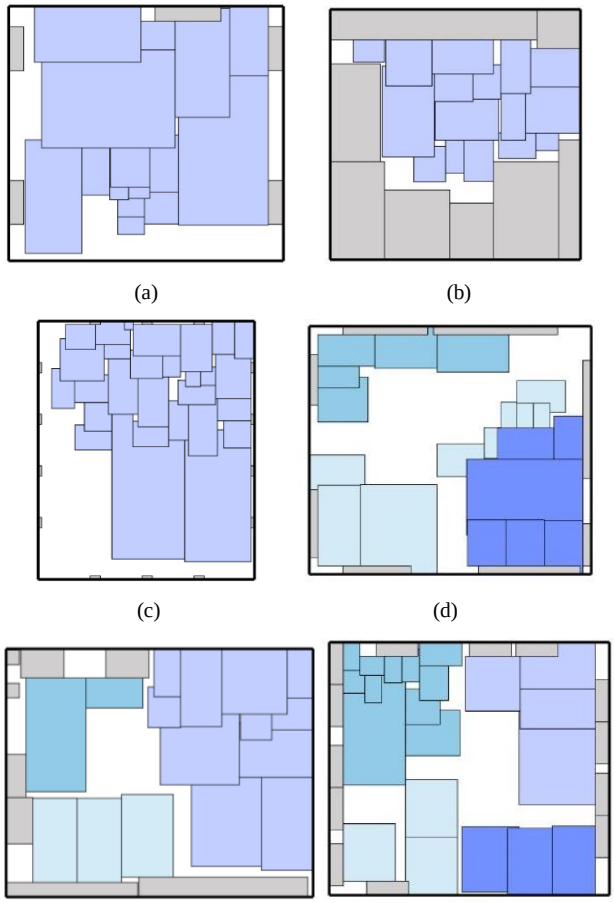
The input data for this experiment is presented in Table I, showing the number of fixed modules, soft modules, and connections within each case, along with relevant information.

Through C++ programming, we successfully placed all modules within a prescribed fixed outline, considering factors such as fixed outline, fixed modules, soft modules, and the irregular shapes soft modules can adopt. Despite these complex conditions, we achieved favorable HPWL values (Table II).

Among them, case04 stands out as one of our best-performing results, surpassing the data provided by ICCAD. Fig. 17 illustrates the placement using MATLAB.

TABLE I. The Input Information for Each Case

Case	Fixed Module	Soft Module	Connection
01	5	15	39236
02	8	16	32953
03	14	28	2750
04	8	20	58575
05	8	16	16600
06	13	21	63500



(a) (b) (c) (d) (e) (f)

Fig. 17. Placement of Visualization. (a) Case01 (b) Case02 (c) Case03 (d) Case04 (e) Case05 (f) Case06

A. The Effectiveness of FM

The experiment compares the method of starting module placement from a selected corner with the method of initially categorizing into four sub-regions using FM, followed by SA for the final layout, with the HPWL results shown in Table III. It's evident that the placement strategy combining FM partitioning and subsequent placement from all four corners yields a shorter HPWL compared to placing modules from only one side. Having a better initial solution can greatly assist in optimizing subsequent results.

TABLE II. Comparison Between Our Achievements and Official Data Released by ICCAD.

Case	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5	Ours	Our Rank
01	152,847,351.00	160,334,303.00	161,567,514.00	162,326,575.00	163,378,965.00	159,411,956.00	**1 - 2**
02	18,632,623.50	20,292,399.00	20,601,431.50	22,688,183.00	23,179,146.00	20,477,630.00	**2 - 3**
03	1,852,003.00	1,888,607.00	1,938,549.00	2,068,568.50	2,135,950.00	2,072,142.00	**4 - 5**
04	63,813,937.50	68,089,725.00	70,873,312.50	72,134,712.50	72,210,275.00	62,103,650.00	**<1**
05	16,058,950.00	16,774,600.00	17,966,700.00	18,632,550.00	19,247,350.00	19,129,000.00	**4 - 5**
06	35,741,550.00	37,895,150.00	40,173,950.00	42,011,500.00	42,644,800.00	38,627,450.00	**2 - 3**

TABLE III. CLASSIFIED FLOORPLANNING RESULTS OF CASE06

Case06	HPWL	HPWL Reduction
FM	38,627,450.00	-
From LD	48,017,100.00	24.31%
From LU	49,627,300.00	28.48%
From RU	46,342,850.00	19.97%
From RD	45,389,900.00	17.51%

B. *The Advantages of Global Placement*

This study examines the benefits of maintaining an undivided chip layout (global placement). By exploring methods that confine modules within an area bounded by invisible cutting lines and aiming to minimize the enclosed area of each sub-region (local placement), the research investigates potential drawbacks. Such placement strategies may result in voids in the chip's center, where modules can still be placed legally but may tend to concentrate in certain areas, leading to suboptimal space utilization, as depicted in Fig. 11. Consequently, it is evident that within the same timeframe, the HPWL of the global placement without cutting line constraints can be reduced by up to 39.32% (Table IV), achieving optimal module placement across the entire chip.

TABLE IV. PARTITIONED FLOORPLANNING RESULTS

Case	Global Placement	Local Placement	HPWL Reduction
01	159,411,956.00	222,087,000.00	39.32%
02	20,477,630.00	25,051,300.00	22.33%
03	2,072,142.00	2,470,400.00	19.22%
04	62,103,650.00	79,185,700.00	27.51%
05	19,129,000.00	22,094,600.00	15.50%
06	38,627,450.00	47,455,100.00	22.85%

C. *Comparison between Fast-SA and Classical-SA*

This experiment investigates the effect of setting the temperature in the SA process. Section IV, particularly subsection E, compares the results of Fast-SA to the traditional approach of multiplying the temperature by a constant slightly less than 1 after each iteration (in this experiment, 0.9 was used).

From Fig. 18, it can be observed that the temperature in the later stages of Classical-SA is limited to very small values, making it prone to reaching local minima in cost. In contrast, Fast-SA exhibits a stage of temperature increase, allowing it to explore other smaller values after uphill moves and avoid being confined to local minima.

The results in Table V also demonstrate that within the same timeframe, Fast-SA can reduce HPWL by up to 146.18% compared to Classical-SA, effectively finding the absolute lowest point in the problem space.

D. *Choosing Parameters for Cost Function*

Based on the experimental data from Fig. 19, we observe that the HPWL varies with different whitespace ratios, depending on the adjustment of the α value. As illustrated, Case 01 to 06 reach their minimum HPWL values at α = 0.7, 0.3, 0.9, 0.9, 0.6, and 0.7, respectively. Therefore, we can infer that when the whitespace ratio is small, the minimum HPWL would more likely be found at small α. While if the whitespace ratio is large, the minimum HPWL would more likely to be found at large α.

VI. CONCLUSION

We propose an algorithm based on our new Quad-Btree and Fast-SA framework to address overlapping and soft module boundary issues arising from fixed outlines and fixed modules in floorplanning. Our research indicates that our Quad-Btree can consistently reduce HPWL more effectively than conventional B*trees, while leveraging Fast-SA achieves faster convergence to desired solutions compared to Classical-SA. Furthermore, we employ global placement instead of local placement for soft modules and adjust the α value of the cost function based on different whitespace ratios to minimize HPWL in each scenario.

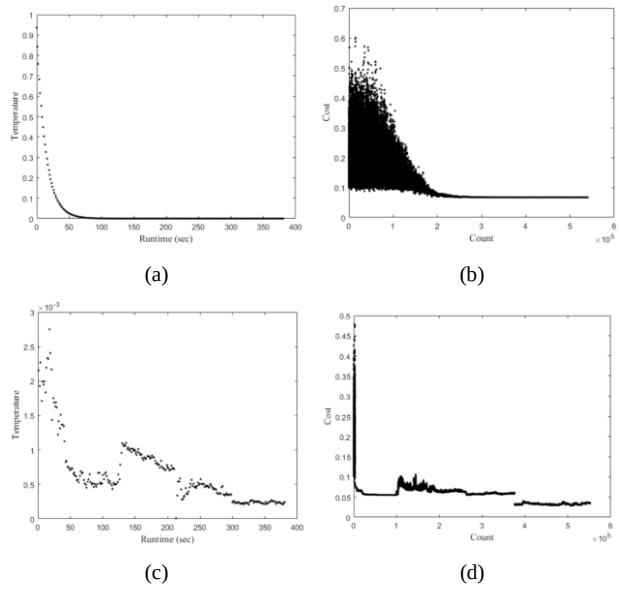


Fig. 18. Temperature and Cost Trends (a) Classical-SA Temperature. (b) Classical-SA Cost. (c) Fast-SA Temperature. (d) Fast-SA Cost.

TABLE V. CLASSICAL-SA AND FAST-SA RESULTS

Case	Fast-SA	Classical-SA	HPWL Reduction
01	159,411,956.00	179,793,193.00	12.79%
02	20,477,630.00	50,411,942.50	146.18%
03	2,072,142.00	3,324,732.00	60.45%
04	62,103,650.00	66,340,112.50	6.82%
05	19,129,000.00	23,564,350.00	23.19%
06	38,627,450.00	47,986,100.00	24.23%

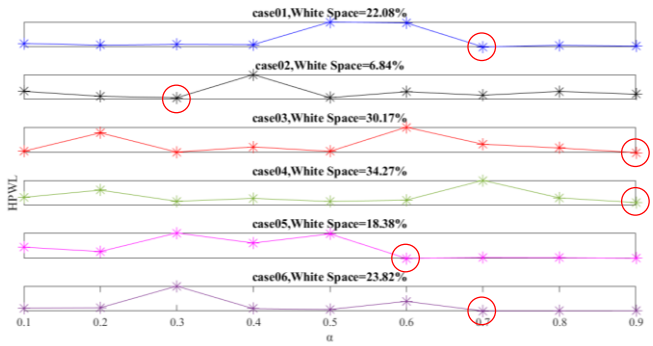


Fig. 19. HPWL Results Obtained with Different α Values

With the increasing complexity of chip design, we anticipate encountering more stringent constraints in the future. We plan to address this by refining our algorithm to

accommodate these challenges and further optimize its performance.

Despite our initial plan to divide all soft modules into four sub-regions, there are instances where we may opt for fewer than four sub-regions (Table VI), especially when modules have tight interconnections. This flexibility in not restricting the number of sub-regions provides our algorithm with significant adaptability to address various scenarios.

TABLE VI. WHITESPACE RATIO AND NUMBER OF LISTS USED

Case	White Space	Regions	HPWL
01	22.08%	1	159,411,956.00
02	6.84%	1	20,477,630.00
03	30.17%	1	2,072,142.00
04	34.27%	3	62,103,650.00
05	18.38%	3	19,129,000.00
06	23.82%	4	38,627,450.00

REFERENCES

[1] Zou, D. X., Hao, G. S., Pan, G., & Wang, G. G. (2015, December). An improved simulated annealing algorithm and area model for the fixed-outline floorplanning with hard modules. In 2015 3rd international symposium on computational and business intelligence (ISCBI) (pp. 21-25). IEEE.

[2] Sheblaev, M. V., & Sheblaeva, A. S. (2018). A method of improving initial partition of Fiduccia–Mattheyses algorithm. Lobachevskii Journal of Mathematics, 39, 1270-1276.

[3] Huang, Z., Lin, Z., Zhu, Z., & Chen, J. (2020). An improved simulated annealing algorithm with excessive length penalty for fixed-outline floorplanning. IEEE Access, 8, 50911-50920.

[4] Comer, D. (1979). Ubiquitous B-tree. ACM Computing Surveys (CSUR), 11(2), 121-137.

[5] Kahng, A. B., Lienig, J., Markov, I. L., & Hu, J. (2011). VLSI physical design: from graph partitioning to timing closure (Vol. 312). Netherlands: Springer.

[6] Tung-Chieh, C., & Yao-Wen, C. (2006). Modern floorplanning based on B*-tree and fast simulated annealing. IEEE Transactions on computer-aided design of integrated circuits and systems, 25(4), 637-650.